

Intrusion Detection Using Neural Networks

De Montfort University Dubai

DMU Dubai Logo

Amir Amhal (P2770828)
Arvin Subramanian (P2940085)
Neeraj Kaushik (P2959114)

School of Computer Science, De Montfort University Dubai

Abstract—This report investigates the use of supervised neural networks for network intrusion detection. Using a curated subset of the KDD’99 family of datasets, we implement a multi-layer perceptron (MLP) with one-hot encoded categorical features and standardized numerical attributes. We address severe class imbalance by targeted undersampling and evaluate performance using accuracy, precision, recall, F1-score and a multi-class confusion matrix. The model achieves near-perfect performance on a held-out test split, while we critically discuss generalization risks, dataset artifacts and avenues for robust, real-world deployment.

Index Terms—Intrusion detection, neural networks, KDD99, NSL-KDD, class imbalance, supervised learning, feature encoding.

CONTENTS

I	Aim	3
II	Objectives	3
III	Literature Review	3
IV	Methodology	3
IV-A	Dataset and Preprocessing	3
IV-B	Model Architecture and Training	3
IV-C	Train/Test Protocol and Metrics	3
V	Practical Implementation and Discussion	3
V-A	Results	3
V-B	Discussion	3
VI	Conclusion and Recommendation	4
	References	4

I. AIM

The aim is to design, train and empirically evaluate a neural-network-based intrusion detection system (IDS) capable of classifying network connections into attack and benign categories with high precision and recall, while documenting a reproducible pipeline from data processing to model evaluation.

II. OBJECTIVES

- 1) Build a clean, end-to-end pipeline for preprocessing, training, and testing on a representative network traffic dataset.
- 2) Develop and evaluate a multi-class neural network to discriminate among frequent attack types and normal traffic.
- 3) Analyze results with appropriate metrics and discuss limitations, threats to validity, and improvements.

III. LITERATURE REVIEW

Intrusion detection has evolved from early rule-based and statistical approaches to machine- and deep-learning techniques. Classical foundations include Denning’s seminal model of intrusion detection [1]. The KDD’99 dataset became a de-facto benchmark, though subsequent work exposed issues such as redundancy and skew [2]. Surveys provide comprehensive overviews of IDS taxonomies, architectures and evaluation practices [3]. With deep learning, feed-forward, recurrent and autoencoder-based approaches have reported strong results on KDD-derived and newer datasets [4], [5], [6]. Meanwhile, more realistic datasets such as UNSW-NB15 address modern traffic characteristics and attack families [7]. Methodologically, advances such as ReLU networks, dropout, Adam optimization and effective software stacks form today’s standard practice [8], [9], [10], [11].

IV. METHODOLOGY

A. Dataset and Preprocessing

We work with a tabular network-connection dataset with 42 columns (features plus label), structurally aligned with KDD’99/NSL-KDD. We focus on five labels with sufficient support and operational relevance: *neptune.*, *normal.*, *smurf.*, *back.*, and *satan.* The raw class distribution is highly imbalanced (e.g., *smurf.* 280,790 samples vs. *satan.* 1,589).

To mitigate skew, we apply targeted undersampling: we select 10,000 examples each for *neptune.*, *normal.* and *smurf.*, while retaining all available examples for minority classes *back.* (2,203) and *satan.* (1,589). The resulting balanced working set contains 33,792 rows. Categorical attributes (e.g., `protocol_type`, `service`, `flag`) are one-hot encoded, and numerical features are standardized to zero mean and unit variance using training-split statistics.

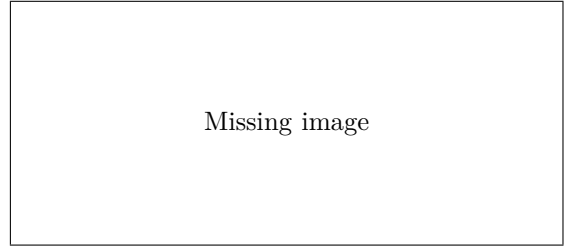


Fig. 1: Example rows and columns after loading the dataset (42 columns).

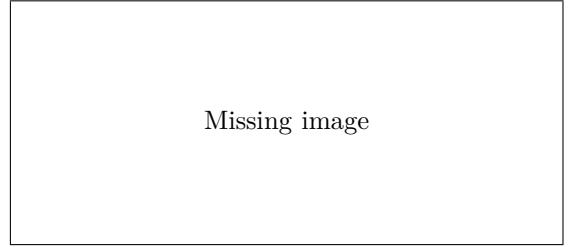


Fig. 2: Full distribution of labels in the source data highlighting severe imbalance.

B. Model Architecture and Training

We adopt a supervised multi-layer perceptron (MLP) classifier with rectified-linear activations, dropout regularization and softmax output for five classes. A representative architecture is: Dense(128)–ReLU → Dropout(0.2) → Dense(64)–ReLU → Dropout(0.2) → Dense(32)–ReLU → Dense(5)–Softmax. The network is trained with categorical cross-entropy loss and Adam optimizer (initial learning rate 10^{-3}), using early stopping on validation loss and mini-batches of size 256. Class weights are not required due to undersampling, but we monitor per-class metrics to detect residual imbalance effects.

C. Train/Test Protocol and Metrics

Data are randomly partitioned into training and test splits with stratification. We evaluate accuracy, macro-averaged precision/recall/F1 and visualize the confusion matrix to understand error modes. All preprocessing steps are fit on the training split only and applied to the test split to prevent leakage.

V. PRACTICAL IMPLEMENTATION AND DISCUSSION

A. Results

The model attains strong performance on the held-out test set (5,069 examples), with overall accuracy ≈ 1.00 . Per-class precision, recall and F1-scores are near perfect, with the minority class *satan.* achieving F1 ≈ 0.99 .

B. Discussion

Successes: Feature engineering with one-hot encoding and standardization combined with undersampling eliminates most class skew, allowing a compact MLP to learn

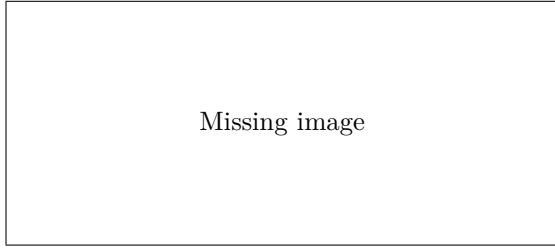


Fig. 3: Final distribution in the balanced working set (33,792 rows).

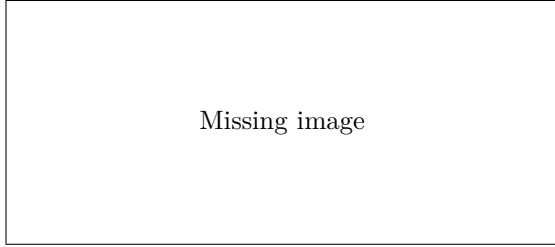


Fig. 4: Multi-class confusion matrix on the test split. Diagonal dominance indicates excellent separability.

highly discriminative representations. The confusion matrix shows minimal cross-class confusions, especially among high-support classes.

Challenges and Threats to Validity: KDD-derived datasets are known to contain redundant records and artifacts that can inflate performance [2]. The train/test split drawn from the same distribution may not reflect deployment scenarios with non-stationary traffic, novel attacks, or adversarial manipulation. Consequently, near-perfect accuracy should be interpreted cautiously. Moreover, undersampling trades recall for majority classes against representativeness of traffic frequencies.

Improvements: Future work should (i) evaluate on harder, modern corpora such as NSL-KDD and UNSW-NB15 [7], (ii) explore cost-sensitive learning and focal loss for residual imbalance, (iii) calibrate probabilities and use detection operating points tuned to security requirements (e.g., high recall at acceptable false positive rate), (iv) perform cross-validation and hyperparameter search, and (v) incorporate explainability (e.g., permutation importance or SHAP) to support analyst trust and triage.

VI. CONCLUSION AND RECOMMENDATION

We demonstrated an end-to-end neural IDS pipeline that achieves near-perfect scores on a balanced subset of a KDD’99-style dataset. The results indicate that modest MLPs can effectively separate common DoS and probe patterns from benign traffic when presented with well-engineered tabular features. For production use, we recommend validating on contemporary, diverse datasets; continuously retraining to handle concept drift; implementing streaming feature extraction and model monitoring; and conducting red-team evaluations to test robustness.

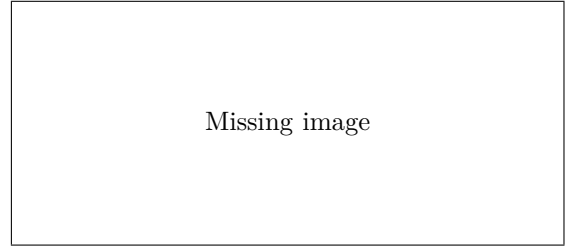


Fig. 5: Classification report summarizing precision, recall, F1-score and support per class.

The approach is readily portable to other intrusion domains (e.g., industrial control networks) with appropriate feature schemas and retraining.

ACKNOWLEDGMENTS

We thank the academic guidance within De Montfort University Dubai and the broader open-source ecosystem supporting reproducible ML research.

REFERENCES

- [1] D. E. Denning, “An intrusion-detection model,” *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, 1987.
- [2] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the kdd cup 99 data set,” in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*. IEEE, 2009, pp. 1–6.
- [3] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, “Intrusion detection system: A comprehensive review,” *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013.
- [4] N. Shone, T. Ngoc Tam, V. D. Phai, and Q. Shi, “A deep learning approach to network intrusion detection,” *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 1, pp. 41–50, 2018.
- [5] C. Yin, Y. Zhu, J. Fei, and X. He, “A deep learning approach for intrusion detection using recurrent neural networks,” *IEEE Access*, vol. 5, pp. 21 954–21 961, 2017.
- [6] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, “A deep learning approach for network intrusion detection system,” in *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS)*, 2016, pp. 21–26.
- [7] N. Moustafa and J. Slay, “Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set),” in *2015 Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015, pp. 1–6.
- [8] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [10] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations*, 2015.
- [11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.